Shravani Anil Patil
D20A - 40

Experiment: 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:
A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:

TrustScore.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract TrustScore {
    // Store trust scores for each user
    mapping(address => uint256) public trustScores;
    address public admin;
    address public loanManagerAddress;
    uint256 public initialScore = 50; // Initial score for new users

    constructor() {
        admin = msg.sender;
    }

    // Set LoanManager address (admin only)
    function setLoanManagerAddress(address _loanManagerAddress) public {
        require(msg.sender == admin, "Only admin");
        loanManagerAddress = _loanManagerAddress;
    }
```

```
    // Admin or LoanManager can update scores
    function updateScore(address user, uint256 score) public {
        require(msg.sender == admin || msg.sender == loanManagerAddress, "Only admin or
LoanManager");
        require(score <= 100, "Score must be 0-100");
        trustScores[user] = score;
    }

    // Set initial score for new users (admin only)
    function setInitialScore(uint256 _initialScore) public {
        require(msg.sender == admin, "Only admin");
        require(_initialScore <= 100, "Score must be 0-100");
        initialScore = _initialScore;
    }

    // Anyone can check a score - returns initialScore if not set
    function getScore(address user) public view returns (uint256) {
        uint256 score = trustScores[user];
        // If score is 0, return initial score (user hasn't been explicitly set)
        return score > 0 ? score : initialScore;
    }
}
```

Deployed TrustScore.sol

LoanManager.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface ITrustScore {
```

```
    function getScore(address user) external view returns (uint256);
    function updateScore(address user, uint256 score) external;
}

contract LoanManager {

    ITrustScore public trustScoreContract;
    address public admin;
    uint256 public minTrustScore;

    // Trust score rewards/penalties
    uint256 public repaymentBonus = 5;       // +5 points for successful repayment
    uint256 public defaultPenalty = 10;      // -10 points for defaulted loan

    // Safety pool address for collecting fees
    address public safetyPoolAddress;

    constructor(address _trustScoreAddress) {
        trustScoreContract = ITrustScore(_trustScoreAddress);
        admin = msg.sender;
        minTrustScore = 40; // Default minimum trust score
    }

    // Set safety pool address (admin only)
    function setSafetyPoolAddress(address _safetyPoolAddress) public {
        require(msg.sender == admin, "Only admin");
        safetyPoolAddress = _safetyPoolAddress;
    }

    struct Loan {
        address borrower;
        address lender;
        uint256 amount;
        uint256 interestRate; // Interest rate in basis points (e.g., 500 = 5%)
        uint256 safetyFee;    // Safety fee in basis points
        uint256 duration;
        uint256 createdAt;
        bool funded;
        bool repaid;
        bool withdrawn;
        bool defaulted;
    }

    Loan[] public loans;
```

```
    event LoanRequested(uint256 loanId, address borrower, uint256 amount, uint256
interestRate);
    event LoanFunded(uint256 loanId, address lender);
    event LoanWithdrawn(uint256 loanId);
    event LoanRepaid(uint256 loanId, uint256 totalRepaid, uint256 interest);
    event LoanDefaulted(uint256 loanId, address borrower);
    event TrustScoreIncreased(address borrower, uint256 amount);
    event TrustScoreDecreased(address borrower, uint256 amount);
    event MinTrustScoreUpdated(uint256 newScore);

    // Calculate interest rate based on trust score
    // Higher trust score = lower interest rate
    function getInterestRate(uint256 trustScore) public pure returns (uint256) {
        if (trustScore >= 90) return 300;      // 3% for excellent credit
        if (trustScore >= 75) return 500;      // 5% for good credit
        if (trustScore >= 60) return 800;      // 8% for fair credit
        if (trustScore >= 50) return 1200;     // 12% for poor credit
        return 1500;                           // 15% for very poor credit
    }

    // Get maximum borrow limit based on trust score (in wei)
    // Higher trust = larger loans allowed
    function getMaxBorrowLimit(uint256 trustScore) public pure returns (uint256) {
        if (trustScore >= 90) return 10 ether;    // Excellent: up to 10 ETH
        if (trustScore >= 75) return 5 ether;     // Good: up to 5 ETH
        if (trustScore >= 60) return 2 ether;     // Fair: up to 2 ETH
        if (trustScore >= 50) return 0.5 ether;   // Poor: up to 0.5 ETH
        return 0.1 ether;                         // Very poor: up to 0.1 ETH
    }

    // Get safety fee based on trust score (in basis points)
    // Lower trust = higher safety fee to protect lenders
    function getSafetyFee(uint256 trustScore) public pure returns (uint256) {
        if (trustScore >= 90) return 50;       // 0.5% for excellent credit
        if (trustScore >= 75) return 100;      // 1% for good credit
        if (trustScore >= 60) return 200;      // 2% for fair credit
        if (trustScore >= 50) return 400;      // 4% for poor credit
        return 600;                            // 6% for very poor credit
    }

    // Get borrower's current limit info
    function getBorrowerLimits(address borrower) public view returns (
        uint256 trustScore,
```

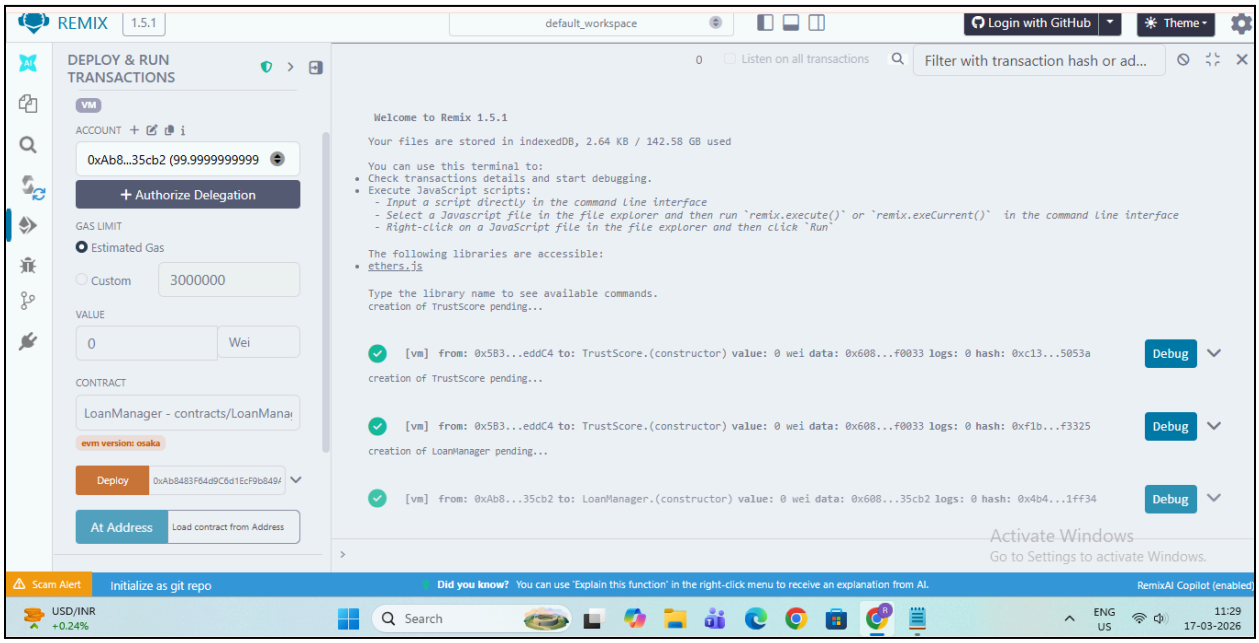
```
    uint256 maxBorrowLimit,
    uint256 interestRate,
    uint256 safetyFee
) {
    trustScore = trustScoreContract.getScore(borrower);
    maxBorrowLimit = getMaxBorrowLimit(trustScore);
    interestRate = getInterestRate(trustScore);
    safetyFee = getSafetyFee(trustScore);
}

// Calculate total repayment amount (principal + interest + safety fee)
function calculateRepayment(uint256 loanId) public view returns (uint256) {
    Loan memory loan = loans[loanId];
    uint256 interest = (loan.amount * loan.interestRate) / 10000;
    uint256 safetyFeeAmount = (loan.amount * loan.safetyFee) / 10000;
    return loan.amount + interest + safetyFeeAmount;
}

// Get breakdown of repayment components
function getRepaymentBreakdown(uint256 loanId) public view returns (
    uint256 principal,
    uint256 interest,
    uint256 safetyFeeAmount,
    uint256 total
) {
    Loan memory loan = loans[loanId];
    principal = loan.amount;
    interest = (loan.amount * loan.interestRate) / 10000;
    safetyFeeAmount = (loan.amount * loan.safetyFee) / 10000;
    total = principal + interest + safetyFeeAmount;
}

// Admin function to update minimum trust score
function setMinTrustScore(uint256 _minScore) public {
    require(msg.sender == admin, "Only admin can set minimum trust score");
    require(_minScore <= 100, "Score must be 0-100");
    minTrustScore = _minScore;
    emit MinTrustScoreUpdated(_minScore);
}

// Set trust score reward for repayment
function setRepaymentBonus(uint256 _bonus) public {
    require(msg.sender == admin, "Only admin");
    repaymentBonus = _bonus;
```

```
    }

    // Set trust score penalty for default
    function setDefaultPenalty(uint256 _penalty) public {
        require(msg.sender == admin, "Only admin");
        defaultPenalty = _penalty;
    }

    // Check if a loan is overdue
    function isLoanOverdue(uint256 loanId) public view returns (bool) {
        Loan memory loan = loans[loanId];
        if (!loan.withdrawn || loan.repaid || loan.defaulted) {
            return false;
        }
        return block.timestamp > loan.createdAt + loan.duration;
    }

    // Mark loan as defaulted (called by admin or lender)
    function markLoanDefaulted(uint256 loanId) public {
        Loan storage loan = loans[loanId];
        require(!loan.defaulted && !loan.repaid, "Loan already finalized");
        require(isLoanOverdue(loanId), "Loan is not overdue");
        require(msg.sender == loan.lender || msg.sender == admin, "Only lender or admin can mark
default");

        loan.defaulted = true;

        // Decrease borrower's trust score
        uint256 currentScore = trustScoreContract.getScore(loan.borrower);
        uint256 newScore = currentScore > defaultPenalty ? currentScore - defaultPenalty : 0;
        trustScoreContract.updateScore(loan.borrower, newScore);

        emit LoanDefaulted(loanId, loan.borrower);
        emit TrustScoreDecreased(loan.borrower, defaultPenalty);
    }

    // ------------------------------
    // BORROWER CREATES LOAN REQUEST
    // ------------------------------
    function createLoan(uint256 amount, uint256 duration) public {

        uint256 score = trustScoreContract.getScore(msg.sender);
        require(score >= minTrustScore, "Trust score too low");
```

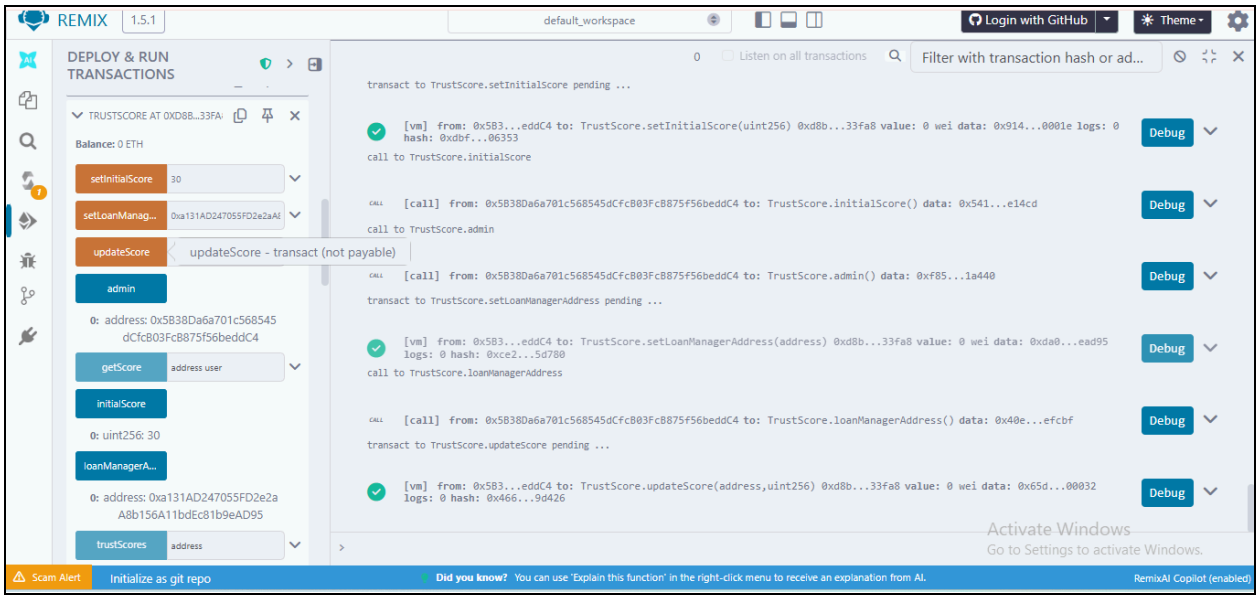
```
    // Enforce maximum borrow limit based on trust score
    uint256 maxLimit = getMaxBorrowLimit(score);
    require(amount <= maxLimit, "Amount exceeds your borrow limit");

    uint256 interestRate = getInterestRate(score);
    uint256 safetyFee = getSafetyFee(score);

    loans.push(
        Loan({
            borrower: msg.sender,
            lender: address(0),
            amount: amount,
            interestRate: interestRate,
            safetyFee: safetyFee,
            duration: duration,
            createdAt: block.timestamp,
            funded: false,
            repaid: false,
            withdrawn: false,
            defaulted: false
        })
    );

    emit LoanRequested(loans.length - 1, msg.sender, amount, interestRate);
}

// ------------------------------
// LENDER FUNDS LOAN
// ------------------------------
function fundLoan(uint256 loanId) public payable {

    Loan storage loan = loans[loanId];

    require(!loan.funded, "Already funded");
    require(msg.value == loan.amount, "Incorrect ETH amount");

    loan.lender = msg.sender;
    loan.funded = true;

    emit LoanFunded(loanId, msg.sender);
}

// ------------------------------
// BORROWER WITHDRAWS FUNDS
```

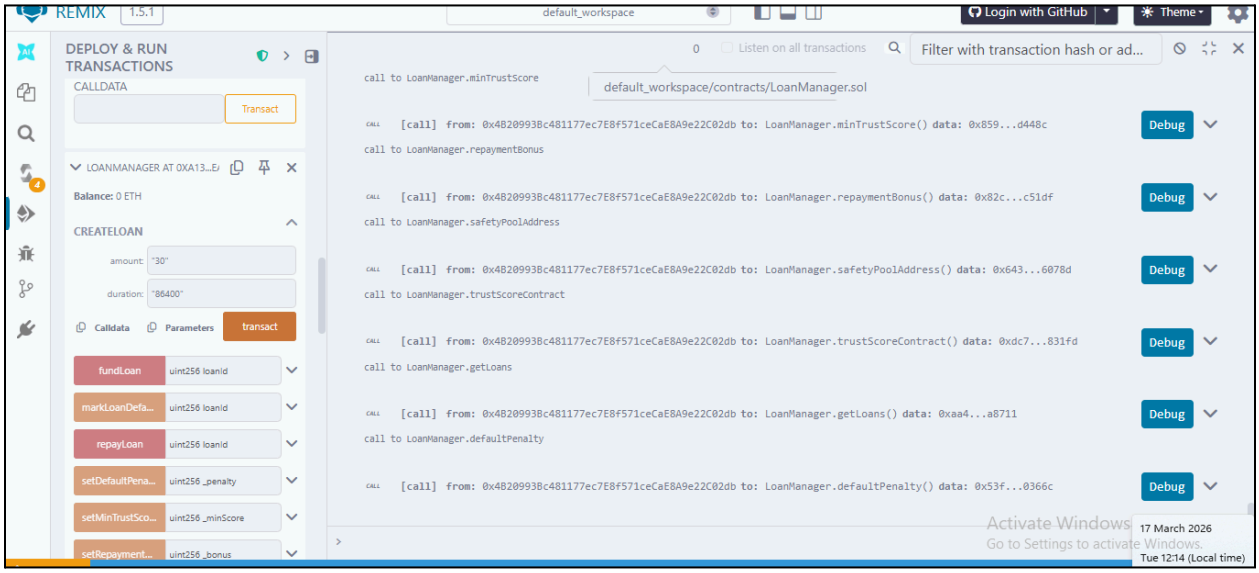
```
    // ------------------------------
    function withdrawLoan(uint256 loanId) public {

        Loan storage loan = loans[loanId];

        require(msg.sender == loan.borrower, "Not borrower");
        require(loan.funded, "Loan not funded");
        require(!loan.withdrawn, "Already withdrawn");

        loan.withdrawn = true;

        payable(msg.sender).transfer(loan.amount);

        emit LoanWithdrawn(loanId);
    }

    // ------------------------------
    // BORROWER REPAYS LOAN
    // ------------------------------
    function repayLoan(uint256 loanId) public payable {

        Loan storage loan = loans[loanId];

        require(msg.sender == loan.borrower, "Not borrower");
        require(loan.withdrawn, "Loan not withdrawn");
        require(!loan.repaid, "Already repaid");
        require(!loan.defaulted, "Loan is defaulted");
        require(loan.lender != address(0), "Invalid lender address");

        // Get repayment breakdown
        (uint256 principal, uint256 interest, uint256 safetyFeeAmount, uint256 totalRepayment) =
getRepaymentBreakdown(loanId);
        require(msg.value >= totalRepayment, "Insufficient repayment amount");

        // Transfer principal + interest to lender
        uint256 lenderAmount = principal + interest;
        payable(loan.lender).transfer(lenderAmount);

        // Transfer safety fee to safety pool (if set)
        if (safetyPoolAddress != address(0) && safetyFeeAmount > 0) {
            payable(safetyPoolAddress).transfer(safetyFeeAmount);
        }

        loan.repaid = true;
```

```
        // Increase borrower's trust score for successful repayment
        uint256 currentScore = trustScoreContract.getScore(loan.borrower);
        uint256 newScore = currentScore + repaymentBonus;
        // Cap at 100
        if (newScore > 100) newScore = 100;
        trustScoreContract.updateScore(loan.borrower, newScore);

        emit LoanRepaid(loanId, totalRepayment, interest);
        emit TrustScoreIncreased(loan.borrower, repaymentBonus);

        // Return excess payment if any
        if (msg.value > totalRepayment) {
            payable(msg.sender).transfer(msg.value - totalRepayment);
        }
    }

    // ------------------------------
    // GET ALL LOANS
    // ------------------------------
    function getLoans() public view returns (Loan[] memory) {
        return loans;
    }
}
```



TrustScore.sol





Conclusion:

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.